

Security Audit

Yieldo (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	12
Conclusion	33
Our Methodology	34
Disclaimers	36
About Hashlock	37

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Yieldo team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Yieldo is a DeFi intelligence and distribution layer that helps users access curated yield with the data to understand it — delivered through the wallets, apps, and curators they already use. The platform combines an independent scoring engine with a white-label SDK and API that lets partners embed institutional-grade yield directly into their products, while end users get transparent risk and performance signals at the point of decision. Attribution is tracked on-chain, allowing vault curators, distribution partners, and creators to share revenue transparently based on the liquidity they originate.

The contracts in scope for this audit form Yieldo's attribution and routing layer. The architecture is deliberately non-custodial and pass-through: user deposits flow atomically into partner vaults within a single transaction, with no protocol-held funds, no fee extraction from deposits, and no discretionary control over user assets. This design minimizes both attack surface and trust assumptions, isolating Yieldo's role to intelligence, attribution, and revenue routing — not custody.

Project Name: Yieldo

Project Type: DeFi

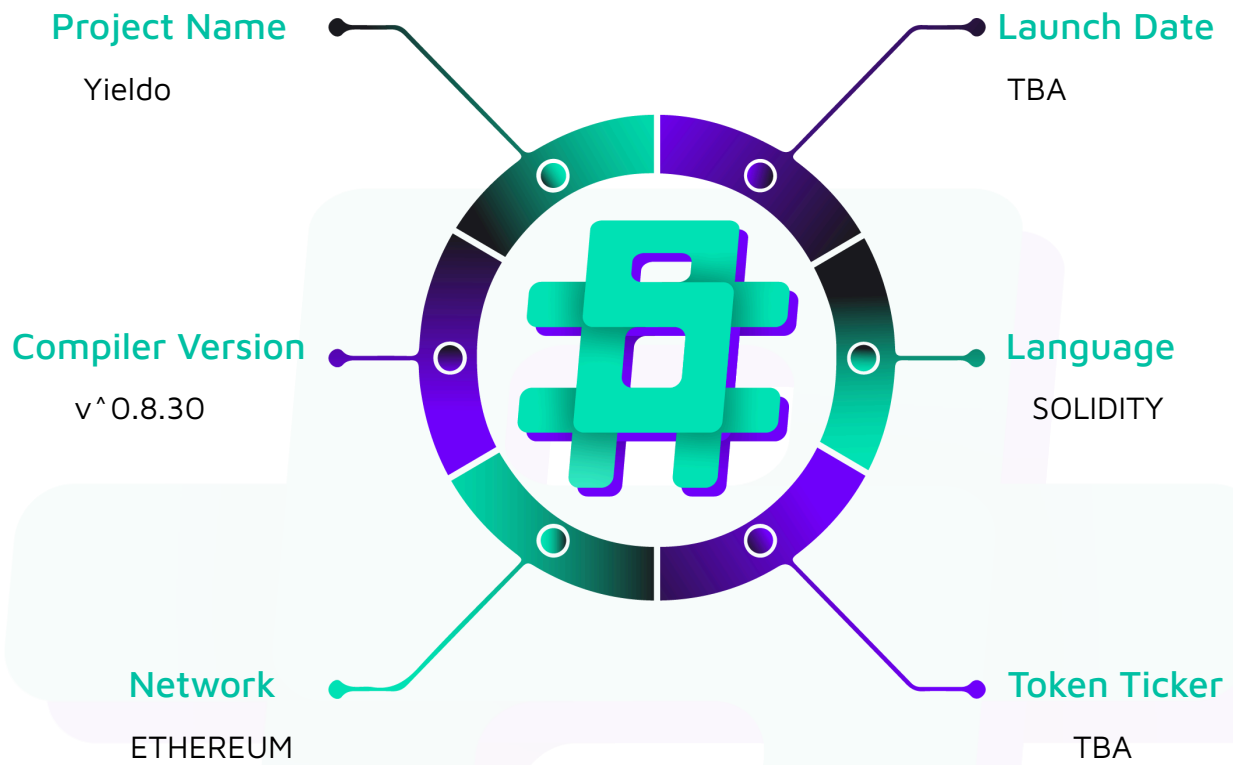
Compiler Version: ^0.8.30

Website: <https://yieldo.xyz/>

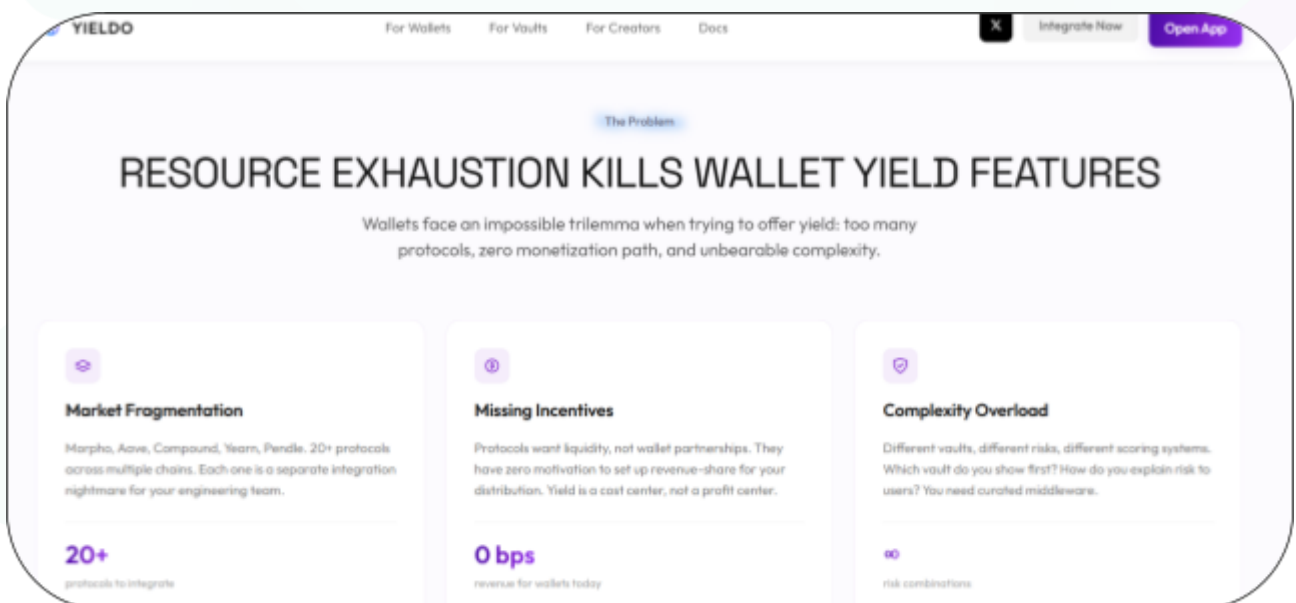
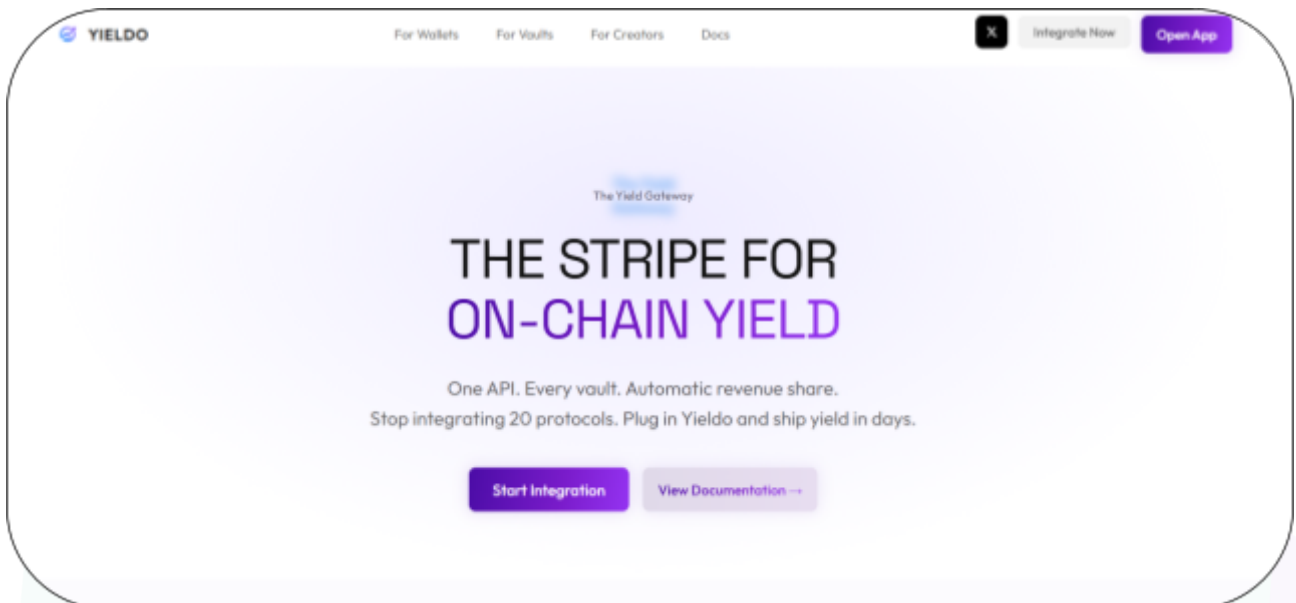
Logo:



Hashlock Pty Ltd

Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Yeldo project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Yeldo Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	May, 2026
Contract 1	DepositRouter.sol
Contract 2	Proxy.sol
Contract 3	IVaultadapter.sol
Audited GitHub Commit Hash	9584a6be37d40c7e16dc56997b3ec7894fbce5a8
Fix Review GitHub Commit Hash	570cc12e4f258c9cd78eadc27e55b7164ce57044

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerability

4 Medium severity vulnerabilities

5 Low severity vulnerabilities

2 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
DepositRouter.sol - Allows the owner to: - Pause and unpause the router - Toggle the vault whitelist - Set allowed vaults - Set Veda tellers - Set Midas vaults - Set Lido deposit queues - Set vault adapters - Set authorized callers - Transfer ownership - Withdraw stuck ETH - Rescue stuck ERC-20 tokens - Authorize implementation upgrades - Allows the pending owner to: - Accept ownership - Allows users to: - Deposit assets to a vault for any recipient with slippage protection - Deposit available allowance and balance for cross-chain composer flows - Submit asynchronous deposit requests - Read the current implementation address	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Yields project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Yields project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] `_executeVaultCall` ERC-4626 path trusts vault-returned shares without verifying recipient balance delta — slippage protection bypass

Description

In the generic ERC-4626 branch of `_executeVaultCall`, the share count used for the `minSharesOut` slippage check is read from the `**vault's return data**` without any cross-check against the recipient's actual share balance delta. The adapter branch directly above explicitly performs this check (`require(delivered >= shares, "Adapter: shares not delivered")`), but the ERC-4626 branch does not. A non-conformant or compromised whitelisted vault can therefore return any `uint256`, defeating the slippage protection and the post-condition `require(shares > 0, ...)`.

Vulnerability Details

`contracts/DepositRouter.sol:340-364`

```
IERC20(asset).forceApprove(vault, amt);
if (isERC4626) {
    uint256 recipBefore4626 = IERC20(vault).balanceOf(recipient);
    (bool vok, bytes memory vrd) = vault.call(
        abi.encodeWithSignature("deposit(uint256,address)", amt, recipient)
    );
    if (!vok) _revertWithReason(vrd, "ERC4626 deposit failed");
    // Most ERC-4626 vaults return shares; some non-standard ones return void.
    // Fall back to balanceOf delta when return data is missing.
    if (vrd.length >= 32) {
        shares = abi.decode(vrd, (uint256)); // <-- trusted blindly
    } else {
        shares = IERC20(vault).balanceOf(recipient) - recipBefore4626;
    }
}
```

`recipBefore4626` is captured but ****never consulted**** when `vrđ.length >= 32`. The downstream slippage gate `require(shares >= minSharesOut, "Slippage")` (line 207) is therefore evaluated against an attacker-controllable value.

Compare against the adapter branch which gets it right:

```
uint256 delivered = IERC20(vault).balanceOf(recipient) - recipBeforeA;
require(delivered >= shares, "Adapter: shares not delivered");
```

The asymmetry is the bug — the same defensive pattern was applied to the trusted-adapter path but omitted from the path that talks to a `_less_` trusted external vault.

The same shape applies to a `_legitimate_` whitelisted vault that gets upgraded or its accounting drifts (oracle manipulation, rounding bug returning an inflated preview): the router accepts the value at face and silently passes the user's `minSharesOut` floor.

Impact

The `minSharesOut` slippage check — the user's primary defense against unfavorable execution — is bypassable by the very contract it is meant to guard against. Loss-of-principal scenarios:

1. Whitelist disabled (`vaultWhitelistEnabled == false`): any caller can craft a target vault that drains assets while reporting any share value.
2. Whitelist enabled but a whitelisted vault is upgradeable (most BoringVault / Yearn-style vaults are): a single compromised upgrade leaks all in-flight `depositFor` calls without tripping the slippage assert.
3. Honest-but-buggy vault: a 4626 implementation with a `previewDeposit`/rounding bug that returns a mis-stated value will produce events and slippage decisions that diverge from on-chain reality.

In all three cases the user pays `amount`, sees a `Routed` event with an attacker-chosen `shares` field, and ends up with fewer (possibly zero) shares.

Recommendation

Mirror the adapter-branch pattern: always compute `delivered = balanceOf(recipient) - recipBefore4626` and `require(delivered >= shares, "...")`. Use `min(shares, delivered)` (or simply `delivered`) as the value compared to `minSharesOut`. Apply the same guard to the `syncDeposit` non-4626 branch's emitted `shares` for consistency (it already uses `delta` — keep it that way).

```
if (vrd.length >= 32) {
    shares = abi.decode(vrd, (uint256));
} else {
    shares = 0;
}
uint256 delivered = IERC20(vault).balanceOf(recipient) - recipBefore4626;
require(delivered >= shares, "Vault: shares not delivered");
shares = delivered; // canonical, attacker-independent
```

Status

Resolved

Medium

[M-01] Routing precedence silently shadows misconfigured vedaTeller / midasVault mappings

Description

`_executeVaultCall` evaluates routing branches in fixed order: `vaultAdapters` → `midasVaults` → `vedaTellers` → `lidoDepositQueues` → generic. If the owner ever populates two of these mappings for the same vault — by mistake, by stale state across upgrades, or because two operators batch different setters in parallel — the higher-priority branch wins silently and the lower-priority configuration is **never reachable**, with no event, no warning, and no on-chain way to detect the shadow.

The risk is not theoretical: `setVedaTeller`, `setMidasVault`, `setVaultAdapter`, `setLidoDepositQueue` each accept the same vault key and never consult their siblings.

Vulnerability Details

`contracts/DepositRouter.sol:264-365`

```
function _executeVaultCall(...) internal returns (uint256 shares) {
    address adapter = vaultAdapters[vault];
    if (adapter != address(0)) { /* ... */ return shares; }    // priority 1

    address midasIV = midasVaults[vault];
    if (midasIV != address(0)) { /* ... */ return shares; }    // priority 2

    address teller = vedaTellers[vault];
    if (teller != address(0)) { /* ... */ return shares; }    // priority 3

    address lidoQueue = lidoDepositQueues[vault][asset];
    if (lidoQueue != address(0)) { /* ... */ return shares; } // priority 4

    // generic ERC4626/syncDeposit branch
}
```

Setters do not enforce mutual exclusion:


```
function setVedaTeller(address vault, address teller) external onlyOwner {
    vedaTellers[vault] = teller;    // no check that vaultAdapters[vault] == 0
    emit VedaTellerUpdated(vault, teller);
}

function setVaultAdapter(address vault, address adapter) external onlyOwner {
    vaultAdapters[vault] = adapter; // silently shadows any pre-existing
    veda/midas/lido entry
    emit VaultAdapterUpdated(vault, adapter);
}
```

Same shape: setting `vaultAdapters[V] = 0` after migrating to Midas is fine, but setting `midasVaults[V] = X` while `vaultAdapters[V] != 0` leaves Midas dead-code.

Impact

- Operator misconfiguration is silently absorbed; the on-chain events suggest both routes are live when only one is reachable.
- After a Veda → adapter migration (or vice-versa), forgetting to zero the previous mapping leaves a "ghost" branch that cannot be detected without reading every mapping.
- During incident response (e.g. pausing a Midas integration), zeroing `midasVaults[V]` does nothing if `vaultAdapters[V]` is also set — the unsafe path keeps executing.

Recommendation

Enforce mutual exclusion at the setter level. Either:

1. In each setter, require that all `_other_` routing mappings for the same `vault` key are zero before allowing the new value. e.g.

```
function setVaultAdapter(address vault, address adapter) external onlyOwner {
    if (adapter != address(0)) {
        require(midasVaults[vault] == address(0), "midas already set");
        require(vedaTellers[vault] == address(0), "veda already set");
    }
    vaultAdapters[vault] = adapter;
    emit VaultAdapterUpdated(vault, adapter);
}
```

2. Or replace the four mappings with a single `mapping(address vault => RouteConfig)` struct holding `(uint8 kind, address target)`, eliminating the precedence question entirely.

Option (2) is the more durable fix; (1) is the minimal-diff fix.

Status

Resolved

[M-02] Non-upgradeable ReentrancyGuard mixed into a UUPS upgradeable contract — storage-layout / upgrade-safety hazard

Description

`DepositRouter` inherits from `@openzeppelin/contracts/utils/ReentrancyGuard.sol` (the non-upgradeable variant) while simultaneously inheriting from `Initializable`, `PausableUpgradeable`, and `UUPSUpgradeable`. OpenZeppelin documentation explicitly warns against this combination. The non-upgradeable `ReentrancyGuard` reserves a regular EVM storage slot (`uint256 _status`) **without** the `__gap` discipline used by the upgradeable mixins, and its constructor (which would set `_status = NOT_ENTERED = 1`) never executes against proxy storage.

Vulnerability Details

`contracts/DepositRouter.sol:7,14`

```
import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
...
contract DepositRouter is Initializable, ReentrancyGuard, PausableUpgradeable,
    UUPSUpgradeable {
```

Two concrete consequences:

1. **Layout fragility.** Under C3 linearization, `ReentrancyGuard._status` occupies a regular storage slot in the contract's layout (after `Initializable`'s ERC-7201 namespaced storage). Any future inheritance reorder, or replacement of `ReentrancyGuard` with `ReentrancyGuardUpgradeable`, will shift every subsequent slot — including the existing `_deprecated_*` fields and the `__gap`. This is exactly the failure mode the lengthy `_deprecated_*` block at `DepositRouter.sol:20-42` was set up to prevent, and a future maintainer would have to manually preserve the slot when migrating to the upgradeable variant.
2. **Constructor-not-run hazard.** OZ v5 `ReentrancyGuard` initializes `_status = 1` in its constructor. For a proxy, the implementation's constructor never runs against proxy storage, so `_status` starts at 0 rather than 1. Functionally the guard still works (`_status != ENTERED(2)` passes), but every first-call costs an extra $0 \rightarrow 2 \rightarrow 1$ SSTORE rather than

1→2→1, and the contract does not match the runtime invariant the OZ team designed against.

Impact

- One-line, well-intentioned cleanup ("use the upgradeable variant for consistency") becomes a catastrophic upgrade.
- Higher gas cost on every first invocation per slot lifecycle (minor, but unnecessary).
- Cognitive trap that a proxy-aware tool like oz-upgrades will flag, but a direct forge create deploy of a new impl will not.

Recommendation

Replace `import "@openzeppelin/contracts/utils/ReentrancyGuard.sol"` with `import "@openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol"`, leave the inheritance position unchanged, and update `initializeV3` to call `__ReentrancyGuard_init()`. Validate the new layout against the deployed proxy with `forge inspect storage-layout` before any next upgrade. If the deployed proxy already has `_status` at a fixed slot, document that slot in the storage map and use `@custom:storage-location` (or a manual slot reservation) to preserve compatibility.

Status

Resolved

[M-03] Asset dust silently retained in router when non-adapter vaults under-pull approved amount

Description

The `vaultAdapters` branch enforces `IERC20(asset).balanceOf(adapter) <= adapterAssetBefore` to guarantee the adapter consumed every wei it received. ****No equivalent invariant exists**** on the Midas, Veda, Lido, generic ERC-4626, or generic `syncDeposit` branches. Each of those uses `forceApprove(target, amt)` and lets the target pull what it wants. If the target pulls only $X < amt$ (rounding floor, deposit cap, paused secondary asset, fee skim), the residual $amt - X$ stays in the router. The user paid `amt`, the slippage gate evaluates against shares-they-got, and the dust is invisible from outside the router.

Vulnerability Details

`contracts/DepositRouter.sol:284-364`

```
// Midas (no asset-residue check)
IERC20(asset).forceApprove(midasIV, amt);
uint256 balBefore = IERC20(vault).balanceOf(address(this));
(bool ok, bytes memory rd) = midasIV.call(
    abi.encodeWithSignature("depositInstant(address,uint256,uint256,bytes32)", asset,
    amt18, 0, bytes32(0))
);
...
shares = IERC20(vault).balanceOf(address(this)) - balBefore;
require(shares > 0, "Midas: zero shares");
IERC20(vault).safeTransfer(recipient, shares);
IERC20(asset).forceApprove(midasIV, 0);
return shares;
// <-- dust = (amt - amount Midas actually pulled) is now stuck in the router

// Veda branch: identical shape, same gap.
// Lido branch: identical shape, same gap.
// Generic ERC4626 branch: same.
// Generic syncDeposit branch: same.
```

Compared with the adapter branch (lines 273-281) which explicitly checks asset-residue.

Impact

User funds become silently strandable in the router. Owner-gated `rescueERC20` (line 182) is the only recovery path — economically: the user paid for shares they didn't receive, and the recovery is at the operator's discretion and timing. Combined with the V3.0 ABI shim that silently forwards `minSharesOut=0`, this is reachable without any malice — just a vault with a temporary cap.

Recommendation

After every external deposit branch, assert the router's asset balance returned to its pre-call value (modulo donations). The cleanest pattern:

```
uint256 routerAssetBefore = IERC20(asset).balanceOf(address(this)) - amt; //
pre-approve baseline
...
// after the external call & share forwarding:
require(IERC20(asset).balanceOf(address(this)) <= routerAssetBefore, "Asset retained
in router");
```

Apply uniformly to Midas / Veda / Lido / ERC-4626 / syncDeposit branches. Optionally refund the residual to `msg.sender` instead of reverting, so callers don't lose the entire tx to a partial-cap deposit.

Status

Resolved

[M-04] Deposit functions lack a deadline parameter — stale-mempool execution risk

Description

`depositFor`, `depositForAvailable`, and `depositRequestFor` accept no `deadline` / expiration. A txn stuck in the mempool (gas underpriced, mempool congestion, or held by a private-orderflow relay) can be mined arbitrarily later, at vault state — share price, deposit cap utilization, fee level — that the user no longer consents to. `minSharesOut` partially compensates but does not bound time, so the same `minSharesOut` set under one volatility regime may be a poor floor under another.

This is especially acute for `depositForAvailable`, which is explicitly designed for cross-chain composer flows where the caller (LiFi Executor) and the eventual mining can be separated by hours.

Vulnerability Details

`contracts/DepositRouter.sol:194-262`

```
function depositFor(..., uint256 minSharesOut) public ...  
function depositForAvailable(..., uint256 minAmount, uint256 minSharesOut) external  
...  
function depositRequestFor(...) external ...
```

No `uint256` `deadline` argument anywhere.

Impact

Loss-of-opportunity / unfavorable execution. Standard finding for any router that takes a slippage parameter without a deadline. Severity is Medium because it does not enable theft, but it materially degrades the slippage protection's usefulness across realistic mempool conditions.

Recommendation

Add a `uint256` `deadline` parameter to all three functions and gate with `require(block.timestamp <= deadline, "Expired")` at the top. Provide a sentinel (e.g.

`deadline == 0` means "no deadline") to ease backwards-compatibility, or — preferred — version-bump the ABI cleanly.

Status

Resolved



Low

[L-01] Initial ownership assignment in `initializeV3` is single-step; pattern conflicts with the rest of the contract's 2-step transfer

Description

`initializeV3` performs a single-shot ownership write (`owner = _owner`) while every subsequent ownership transfer uses a 2-step pattern (`transferOwnership` → `acceptOwnership`). If `_owner` is mistyped at deploy/upgrade time — common with multisig address copy-paste mistakes — there is no recovery path other than redeploying.

Vulnerability Details

`contracts/DepositRouter.sol:83-87`

```
function initializeV3(address _owner) external initializer {
    require(_owner != address(0), "Zero owner");
    __Pausable_init();
    owner = _owner; // direct assignment, no 2-step}
```

VS.

```
function transferOwnership(address newOwner) external onlyOwner { pendingOwner =
newOwner; ... }
function acceptOwnership() external { require(msg.sender == pendingOwner, ...); owner
= pendingOwner; ... }
```

Recommendation

Either (a) mirror the 2-step pattern at init by writing `pendingOwner = _owner` and requiring the new owner to call `acceptOwnership`, or (b) accept a separate `_initialPendingOwner` and require an explicit acceptance after deploy. Option (a) keeps the ABI simple.

Status

Resolved

[L-02] `initializeV3` is front-runnable on a freshly-deployed proxy without atomic init

Description

`initializeV3` uses the OZ v5 `initializer` modifier, which is callable once on a proxy where `_initialized == 0`. `YieldoProxy`'s constructor accepts a `data` blob and forwards it as a delegatecall, so atomic initialization is `_available_` — but not enforced. If a deployment script ever deploys `YieldoProxy(impl, "")` (empty data) and runs `initializeV3` in a follow-up transaction, an attacker who watches the mempool can call `initializeV3(attacker)` first and become permanent owner.

Vulnerability Details

contracts/Proxy.sol:6-8

```
contract YieldoProxy is ERC1967Proxy {
    constructor(address impl, bytes memory data) ERC1967Proxy(impl, data) {}
}
```

contracts/DepositRouter.sol:83

```
function initializeV3(address _owner) external initializer { ... }
```

ERC1967Proxy's constructor only delegatecalls into `impl` if `data.length > 0`. An empty data produces an uninitialized proxy.

Recommendation

Document and enforce in the deployment runbook that `YieldoProxy` MUST be deployed with non-empty data containing the abi-encoded `initializeV3(_owner)` call. Optionally tighten `YieldoProxy` to require `data.length > 0` at construction:

```
constructor(address impl, bytes memory data) ERC1967Proxy(impl, data) {
    require(data.length > 0, "Init data required");
}
```

Status

Resolved

[L-03] Setters do not validate that the new state differs from the old state

Description

setVaultWhitelistEnabled, setVaultAllowed, setVedaTeller, setMidasVault, setLidoDepositQueue, setVaultAdapter, setAuthorizedCaller, and their batch counterparts unconditionally write the new value and emit the corresponding event. A no-op call therefore emits a misleading event and can pollute downstream indexers / monitoring.

Vulnerability Details

contracts/DepositRouter.sol:104-169

```
function setVaultWhitelistEnabled(bool _e) external onlyOwner {
    vaultWhitelistEnabled = _e;
    emit VaultWhitelistToggled(_e);
}

function setVaultAllowed(address v, bool a) external onlyOwner {
    allowedVaults[v] = a;
    emit VaultAllowlistUpdated(v, a);
}

// ... and 6 more in the same shape
```

Recommendation

Add `require(currentValue != newValue, "No-op")` at the top of each setter, OR cheaply early-return when the value is unchanged. Consistent treatment across all setters keeps event-driven off-chain state coherent.

Status

Resolved

[L-04] Lido `referrer` argument hardcoded to `address(0)` — abandons potential referral economics

Description

The Lido `SyncDepositQueue` branch calls `deposit(uint224 amt, address referrer, bytes32[] proof)` with `referrer = address(0)`. If Yieldo is eligible for Lido referral rewards (current or future), every deposit through this path forfeits them.

Vulnerability Details

`contracts/DepositRouter.sol:329-331`

```
(bool ok, bytes memory rd) = lidoQueue.call(
    abi.encodeWithSignature("deposit(uint224,address,bytes32[])", uint224(amt),
    address(0), emptyProof)
);
```

Recommendation

Add a Yieldo-controlled referrer address to storage (settable by owner, defaulting to `address(0)` for backward compatibility) and pass it instead of `address(0)`. If referral economics are not relevant for `SyncDepositQueue` today, document the choice in a comment so future maintainers don't overlook it.

Status

Resolved

[L-05] Fee-on-transfer and rebasing assets revert opaquely; not documented

Description

`safeTransferFrom(msg.sender, address(this), amount)` followed by `safeTransfer(adapter, amt)` (or `forceApprove(spender, amt)`) assumes `amount` survives transit. Fee-on-transfer tokens leave the router with `amount * (1 - fee)`, after which the next transfer to the adapter / vault will revert with "ERC20: transfer amount exceeds balance". Rebasing tokens have similar timing-dependent failures. The contract does not document this exclusion, and the revert is not informative.

Vulnerability Details

`contracts/DepositRouter.sol:205, 244, 259, 275`

Recommendation

Either (a) add a balance-delta check after the inbound `safeTransferFrom` and reject FoT assets with a clear error, or (b) document in the contract NatSpec that only standard ERC-20 (`balanceOf` mirrors `transfer` amount, no rebases) is supported and silently inherit the underlying revert. Option (a) is preferable for ops UX.

Status

Resolved

QA

[Q-01] `authorizedCallers` mapping and setters are non-functional

Description

`authorizedCallers` and `setAuthorizedCaller` / `setAuthorizedCallerBatch` are kept "for potential future re-enablement" (per the in-code comment at `DepositRouter.sol:188-193`) but no code path consults the mapping. Storage and an `AuthorizedCallerUpdated` event keep being written, with no on-chain effect. Off-chain integrations / explorers may misinterpret the events as enforcing access.

Recommendation

Either (a) re-introduce the consultation in `depositFor` / `depositForAvailable` / `depositRequestFor` behind a feature flag (e.g. an `authChecksEnabled` bool), or (b) delete the setters and stop emitting `AuthorizedCallerUpdated` until the feature actually returns. Leaving misleading events is the worst of both worlds.

Status

Resolved

[Q-02] `VERSION` is a string public constant rather than a function**Description**

Using a string public constant for `VERSION` puts the version in implementation bytecode — fine, but each upgrade requires a code change to bump it, and there is no on-chain way to differentiate "VERSION reads 3.2.0 because that's the deployed impl" from "VERSION reads 3.2.0 because nobody updated the constant". A pure function returning the same value, paired with a human review of `getImplementation()` against expected addresses, is mildly more reliable.

Recommendation

Optional: replace with `function version() external pure returns (string memory) { return "3.2.0"; }`. If kept as-is, ensure the constant is mechanically updated on every impl deployment via the release process.

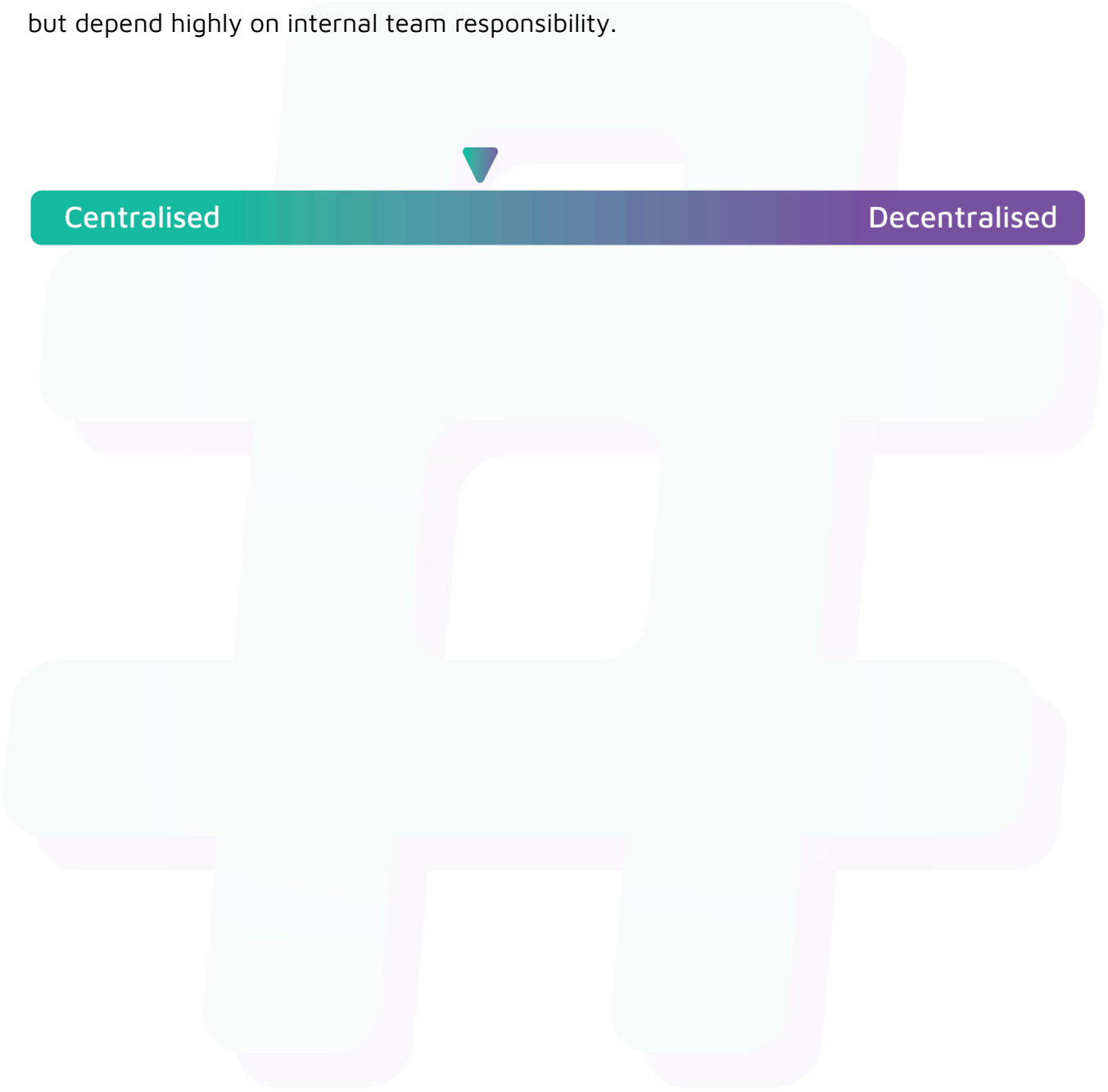
Status

Acknowledged

Centralisation

The Yieldo project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Yieldo project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.